

Imperative Languages

Programming Languages Intro

Dr. Igor Steinmacher

Imperative Programming

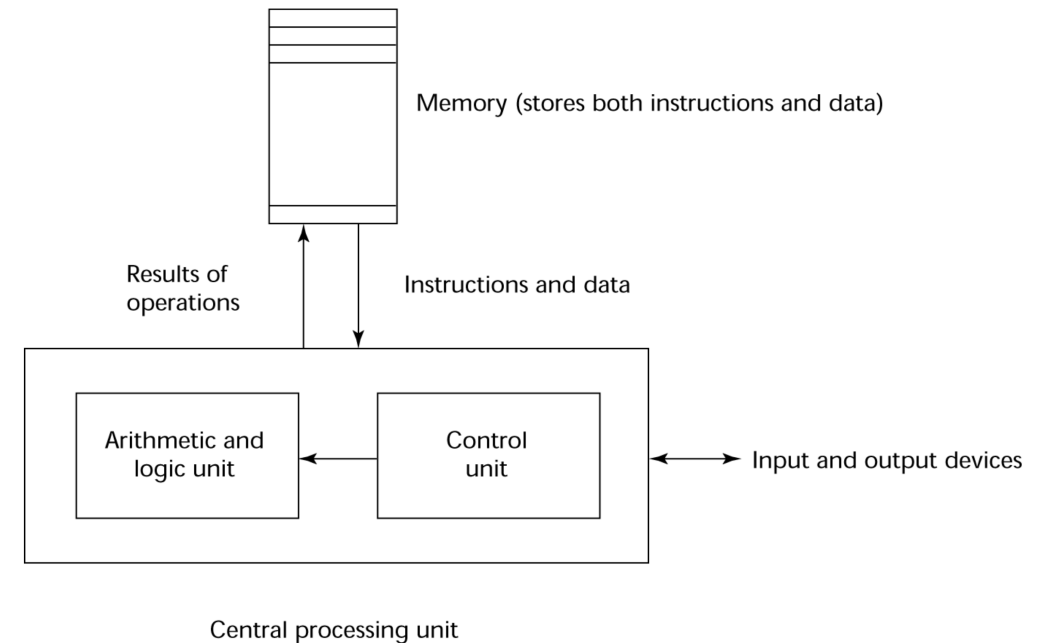
- Oldest and most well-developed paradigm
- Mirrors computer architecture
- Known Languages
 - Fortran, Pascal
 - C, Clite
 - Ada 83
 - Perl

Imperative Programming: Procedural

- **Procedural abstraction:** programmer says how the computer needs to work (Imperative!)
 - concerned mainly with a function interface, ignoring the details of how it is computed
- Focus on computer efficiency
- The process of **refinement** utilizes **procedural** abstraction to develop an algorithm starting with a general form and ending with an implementation

Imperative Programming Characteristics

- In a von Neumann machine memory holds:
 - Instructions
 - Data
 - Intellectual heart: assignment statement
 - Others:
 - Conditional branching
 - Unconditional branch (goto)



Imperative Programming Characteristics

- **Key operation: Assignment (=)** is fundamental
 - target = expression
 - Side effect: updating state (i.e., memory) of the machine
- **Reference semantics**
 - Expression is evaluated to an object, whose pointer is copied to the target (used by object-oriented languages)

Imperative Programming Characteristics

- Turing Completeness
 - Assignment
 - Sequence
 - Conditional: If
 - Loop:
 - Goto
 - Replaced by while loops

Imperative Programming Characteristics

- Data type definitions
- Variable declarations (usually typed)
- Expressions and assignment statements
- Control flow statements (usually structured)
- Lexical scopes and blocks
 - Goal: provide locality of reference
- Declarations and definitions of procedures and functions (i.e., parameterized blocks)

Extending the procedures

- Several libraries of functions for most imperative languages
- Longevity/improvements of languages like Fortran, Cobol, and C
- Typical libraries for data structures
 - Iterators
 - Vectors
 - Lists
 - Stacks, queues, dequeues, priority queues
 - Sets and bags
 - Maps

Example of Imperative Language: C

- “low-level” language
- Conditional compilation
- Lacks: iterators, generics, exception handling, overloading
- Assignments are expression
 - ex: *strcpy*
- *Dynamic Allocation:*
*int *a;*
*a = malloc(sizeof(int) *size);*

Average Code in C

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    int ct, number, min, max, sum;
    sum = ct = 0;
    printf("Enter number: ");
    while (scanf("%d", number) != EOF) {
        if (ct==0) min = max = number;
        ct++;
        sum += number;
        min = number < min? number : min;
        max = number > max? number : max;
        printf("Enter number: ");
    }
    printf("%d numbers read\n", ct);
    if (ct>0){
        printf("Average: \t%d\n", sum / ct);
        printf("Maximum:\t%d\n", max);
        printf("Minimum: \t%d\n", min);
    }
}
```

- Compute min, max, average of a set of numbers
- **Formatting (in/out):** scanf, printf
- **Conditional** assignment
- 2nd argument to scanf must be an address
 - caught by some compilers
 - segment violation at run time

**See you in our next
Lesson!**